# Software Architecture and Design Specification (SAD)

*Remote Lab Reservation System*

| | |
|---|---|
| **Project** | Remote Lab Reservation System (RLRS) |
| **Version** | 1.0 |
| **Authors** | Four-member Software Engineering project team |
| **Date** | 30 October 2026 |
| **Status** | Draft for team review |

## Revision History

| Version | Date | Author | Change summary | Approval |
|---|---|---|---|---|
| 1.0 | 30 October 2026 | Four-member Software Engineering project team | Initial project-specific draft | Pending |

## Approvals

| Role | Name | Signature / Email | Date |
|---|---|---|---|
| Project team representative | To be filled by team | Pending | |
| Course instructor / TA | To be filled by instructor | Pending | |

## Table of Contents


- 1. Introduction
- 2. Architecture Goals and Constraints
- 3. Stakeholders and Concerns
- 4. Architecture and Components
- 5. Architecture Pattern and Technology Stack
- 6. Data Design
- 7. API / Service Design
- 8. Sequence Flows
- 9. Security Architecture
- 10. Error Handling, Logging and Monitoring
- 11. Traceability, Risks and Open Issues


# 1. Introduction

## 1.1 Purpose

This document describes the proposed architecture and design for RLRS. It defines component responsibilities, data relationships, the scheduling engine and its conflict-detection rules, the RESTful API, the email reminder pipeline, important flows, security controls, and traceability to the SRS.

## 1.2 Scope

The design covers lab and equipment management, operating hours and slot configuration, availability calendars, reservation creation, approval and cancellation, conflict detection and booking rules, maintenance blocks, confirmation and reminder emails, attendance, reports, authentication, authorization, and audit history. Direct equipment control, timetable/SIS integration, SSO, iCal export and SMS are optional future work.

# 2. Architecture Goals and Constraints

- No double booking: conflict detection must hold even when requests arrive at the same time.
- Keep all scheduling and booking-rule logic in one scheduling engine that can be unit-tested without the UI, web server or email service.
- Send each reminder exactly once, at the right time, and survive restarts without losing or duplicating emails.
- Use a small, understandable architecture suitable for a four-member student team and sprint delivery.
- Match final diagrams and technology descriptions to the implementation; record deviations before submission.

# 3. Stakeholders and Concerns

| Stakeholder | Primary concern |
|---|---|
| Student | Seeing real availability quickly, booking in a few clicks, reliable reminders. |
| Faculty | Booking equipment for sessions without clashes. |
| Lab in-charge | Accurate schedules, control over restricted equipment and maintenance, attendance records. |
| Administrator | User access, booking rules, email configuration, auditability. |
| Developers | Modular code, testable scheduling engine, clear REST contracts. |
| Evaluator / TA | Traceability from requirements to implementation, tests, commits, and demos. |

# 4. Architecture and Components

## 4.1 Chosen Pattern

Use a layered modular-monolith architecture: a browser front end with a calendar widget, a REST API layer, business services (including the scheduling engine), repositories/data-access, and a relational database. A background job scheduler runs in a separate worker process from the same codebase to send emails. This keeps deployment and collaboration simple while isolating time-based work from web requests. A microservices architecture is not recommended for the initial prototype because it would add operational overhead without a demonstrated need.

## 4.2 Component Responsibilities

| Component | Responsibilities | Key interactions |
|---|---|---|
| Front end / calendar UI | Day/week calendar widget, booking form, my-reservations page, lab in-charge screens, error display. | Calls the REST API; renders availability and results. |
| REST API layer | Routes, request validation, JSON serialisation, HTTP status codes. | Calls Auth and business services. |
| Authentication / authorization | Sign-in/out, session or token validation, role checks, record-level ownership checks. | Protects every API route and service operation. |
| Lab & equipment service | CRUD for labs and equipment; operating hours and slot length. | Uses lab/equipment repositories; emits audit events. |
| Availability service | Builds the slot grid for a date range and marks each slot free, booked, own or maintenance. | Reads equipment, reservations and maintenance blocks. |
| Scheduling engine | Conflict detection and booking-rule checks; atomically creates, cancels and re-validates reservations. | Uses reservation repository inside a database transaction. |
| Reservation service | Reservation lifecycle: create, approve/reject, cancel, mark attendance. | Delegates every booking decision to the Scheduling engine; calls Notification and Audit services. |
| Notification service | Creates email jobs (confirmation, cancellation, reminder) and renders templates. | Writes EmailJob rows; read by the Job worker. |
| Job scheduler / worker | Periodically picks due email jobs, sends them via SMTP, retries failures, marks results. | Reads/updates EmailJob; talks to the SMTP service. |
| Reporting service | Utilisation, booking, cancellation and no-show reports. | Queries reservations, equipment and operating hours. |
| Audit service | Record important actions and actor/timestamp details. | Called by all mutating services. |
| Repository / data-access layer | Encapsulate database queries, transactions and locking. | Used by business services and the worker. |
| Relational database | Persist users, labs, equipment, reservations, maintenance blocks, email jobs, settings and audit events. | Accessed only through data-access layer. |

## 4.3 Component Diagram (text specification)

Browser (Calendar UI) → REST API → (Authentication & Authorization; Lab & Equipment Service; Availability Service; Reservation Service → Scheduling Engine; Notification Service; Reporting Service; Audit Service) → Repositories → Relational Database. Job Worker → EmailJob repository → Relational Database, and Job Worker → SMTP Service. Reservation Service calls Notification Service and Audit Service after each successful change. Availability and Reporting services only read data.

# 5. Architecture Pattern and Technology Stack

Use the technologies already selected in the repository. The proposed implementation is a Python REST API (for example FastAPI or Flask), a relational database (PostgreSQL preferred because it supports range exclusion constraints; SQLite acceptable for local development), a job scheduler such as APScheduler or Celery beat for the

email worker, SMTP for email (Mailpit or a similar local mail catcher in development), and a browser front end using a calendar widget library such as FullCalendar. The precise choices should be recorded from the actual repository rather than assumed in this draft.

Development tooling: Git/GitHub branches and pull requests, automated checks through GitHub Actions, unit tests for the scheduling engine and services, and a README describing environment setup and required environment variables. Production hosting and external integrations are outside the initial scope.

## 6. Data Design

| Entity | Important fields | Relationships / rules |
|---|---|---|
| User | id, name, email, password_hash, role_id, active | Role assigned to each user; email is unique; never store plaintext password. |
| Role | id, name, permissions | Controls protected operations (Admin, Lab In-charge, Faculty, Student, Viewer). |
| Lab | id, name, location, open_time, close_time, slot_minutes, in_charge_id, time_zone | One lab has many equipment items; slot_minutes defines the calendar grid. |
| Equipment | id, lab_id, name, type, description, requires_approval, remote_access_info, active | Belongs to one lab; remote_access_info shown only to confirmed holders. |
| Reservation | id, equipment_id, user_id, start_at, end_at, status, purpose, rejection_reason, created_at, cancelled_at | start_at < end_at; no overlap with another Pending/Confirmed reservation on the same equipment. |
| MaintenanceBlock | id, equipment_id, start_at, end_at, reason, created_by | No reservation may be Pending/Confirmed inside a block. |
| EmailJob | id, reservation_id, user_id, type, send_at, status, attempts, last_error, sent_at | Unique (reservation_id, type) for reminders so each is sent once. |
| BookingRule / SystemSetting | key, value, updated_by, updated_at | Advance window, max duration, max active bookings, cancellation cut-off, reminder lead time. |
| AuditEvent | id, user_id, action, entity_type, entity_id, timestamp, summary | No passwords or secrets in event payloads. |

### 6.1 Scheduling Engine and Conflict Detection

Two time ranges [s1, e1) and [s2, e2) conflict when s1 < e2 and s2 < e1. Ranges are half-open, so a booking ending at 10:00 does not conflict with one starting at 10:00. For a new request the engine runs these checks in order and returns the first failure:

- Equipment exists and is active; the user is allowed to book it.
- Start and end fall on slot boundaries, start is in the future, and the range lies inside the lab's operating hours.
- Start is within the advance-booking window and duration does not exceed the maximum.
- The user's active (Pending/Confirmed) future reservations are below the limit, and none of them overlaps the requested range.
- No Pending/Confirmed reservation and no maintenance block on the same equipment overlaps the requested range.

The final overlap check and the insert run inside one database transaction. With PostgreSQL, an exclusion constraint on (equipment_id, tstzrange(start_at, end_at)) for active statuses guarantees no double booking even under concurrent requests. On databases without range constraints, the engine locks the equipment row (SELECT ... FOR UPDATE) before checking and inserting.

### 6.2 Reservation State Model

| From status | Allowed next status | Who / trigger |
|---|---|---|
| (new) | Confirmed or Pending | Scheduling engine; Pending if equipment requires approval |
| Pending | Confirmed, Rejected or Cancelled | Lab in-charge approves/rejects (reason required); user cancels |
| Confirmed | Cancelled | User before cut-off; lab in-charge or |

| | | |
|---|---|---|
| | | maintenance block at any time |
| Confirmed | Completed or No-show | Lab in-charge after the slot ends |
| Rejected / Cancelled / Completed / No-show | None (terminal) | Slot is free again for Rejected and Cancelled |

### 6.3 Data Integrity Rules

- Use database constraints (foreign keys, NOT NULL, CHECK start_at < end_at, exclusion or locking) and repeat the rules in the scheduling engine.
- Store timestamps in UTC; convert to the lab's time zone for operating hours, slot boundaries and display.
- Create EmailJob rows in the same transaction as the reservation change that causes them.
- When a reservation is cancelled or rejected, mark its pending reminder EmailJob as Cancelled in the same transaction.
- Keep AuditEvent append-only.

## 7. API / Service Design

The exact URL routing may follow the repository's framework. The following is a proposed RESTful contract; adapt it to the implementation and document any differences. All bodies are JSON.

| Operation | Method / route (example) | Input | Successful result / errors |
|---|---|---|---|
| List labs / equipment | GET /api/labs; GET /api/equipment | lab_id, type, available_on | List; 400 for invalid filter. |
| Manage equipment | POST /api/equipment; PATCH /api/equipment/{id} | name, lab_id, type, requires_approval, remote_access_info, active | Created/updated; 400 validation; 403 not lab in-charge. |
| Get availability | GET /api/equipment/{id}/availability | from, to (max 31 days) | Slot list with state; 400 invalid range. |
| Create reservation | POST /api/reservations | equipment_id, start_at, end_at, purpose | 201 with status Confirmed/Pending; 400 rule violation; 409 conflict with details. |
| My reservations | GET /api/reservations?mine=true | status, from, to | User's reservations. |
| Cancel reservation | POST /api/reservations/{id}/cancel | reason (optional) | Status Cancelled; 403 not owner; 409 past cut-off or not cancellable. |
| Approve / reject | POST /api/reservations/{id}/approve; /reject | reason (reject) | Status updated; 400 missing reason; 403 not lab in-charge; 409 no longer Pending. |
| Mark attendance | POST /api/reservations/{id}/attendance | attended (true/false) | Completed or No-show; 409 slot not finished. |
| Maintenance block | POST /api/maintenance-blocks | equipment_id, start_at, end_at, reason | Block created; affected reservations cancelled and notified. |
| Settings / reports | PUT /api/settings; GET /api/reports/{type} | rule values; date range and filters | Updated settings or report data; 400 invalid; 403 unauthorized. |

## 8. Sequence Flows

### 8.1 Create Reservation

- Student selects a free slot in the calendar; the booking form submits equipment, start, end and purpose.
- Front end sends POST /api/reservations; the API layer validates the payload and checks authentication.
- Reservation Service calls the Scheduling Engine, which opens a transaction and runs the rule and conflict checks.
- If all checks pass, the engine inserts the reservation (Confirmed or Pending) and the confirmation and reminder EmailJob rows, then commits.

- If the conflict check or the database constraint fails, the transaction is rolled back and the API returns 409 with the conflicting time range.
- Audit Service records the booking; the calendar refreshes and shows the slot as the user's own booking.

### 8.2 Send Reminder

- The Job worker runs every minute and selects EmailJobs with status Queued and send_at in the past, locking each row it picks.
- For each job, it re-reads the reservation; if the reservation is no longer Confirmed, the job is marked Cancelled.
- Otherwise it renders the reminder template and sends it through SMTP.
- On success the job is marked Sent with sent_at; on a temporary failure attempts is incremented and send_at is pushed back; after the maximum attempts the job is marked Failed.
- Because the job status is updated in the same transaction as the lock, a restarted or second worker never sends the same reminder twice.

### 8.3 Maintenance Block

- Lab in-charge submits a maintenance block for an equipment item and time range.
- The engine finds Pending/Confirmed reservations that overlap the block, cancels them, and cancels their reminders.
- A cancellation EmailJob is queued for each affected user, all in one transaction.
- The calendar shows the range as maintenance and new bookings inside it are rejected.

## 9. Security Architecture

| Threat / concern | Control |
|---|---|
| Unauthorized access | Authentication plus server-side role checks for every protected API route. |
| Acting on someone else's booking | Record-level checks: only the owner (or the lab in-charge/admin) can cancel; lab in-charges only manage their own labs. |
| Double booking / slot grabbing | Transactional conflict check plus database exclusion constraint or row lock; per-user active-booking limit. |
| Credential and secret disclosure | Salted password hashing, HTTPS in deployed environments, SMTP credentials in environment variables, secret-free logs. |
| Exposure of remote-access details | Returned only to the holder of a confirmed reservation for that equipment. |
| Injection / cross-site scripting | Parameterized database access, strict input validation, output escaping in the UI and in email templates. |
| Privilege escalation | Deny-by-default permissions and tests for direct requests by unauthorized roles. |
| Abuse of booking or email endpoints | Request rate limiting and limits on date ranges and active bookings. |

Threat-modeling prompts: spoofing (fake user/session), tampering (editing another user's booking or bypassing conflict checks), repudiation (denial of a cancellation or no-show), information disclosure (remote-access details or other users' bookings exposed), denial of service (mass booking or excessive availability queries), and elevation of privilege (student performing lab in-charge or admin operations).

## 10. Error Handling, Logging and Monitoring

- Return consistent validation (400), permission (403), not-found (404) and conflict (409) errors with a readable message; do not show stack traces to end users.
- Log unexpected failures with a correlation/reference ID where feasible; never log passwords, tokens or SMTP secrets.
- Record business audit events separately from diagnostic logs.
- Monitor rejected bookings by reason, constraint violations, email jobs in Failed status, worker heartbeat, and the gap between scheduled and actual send time.
- Provide an understandable empty state when no equipment, slots or reservations match the selected filter.

## 11. Traceability, Risks and Open Issues

| SRS requirement | Design component |
|---|---|
| RLRS-F-001, F-002 / RLRS-SR-001, SR-002 | Authentication / authorization |
| RLRS-F-003 to RLRS-F-005 | Lab & Equipment Service |
| RLRS-F-006 | Availability Service and calendar UI |
| RLRS-F-007 to RLRS-F-009 / RLRS-NF-002 | Scheduling Engine, exclusion constraint / row lock |
| RLRS-F-010 to RLRS-F-012, F-018 | Reservation Service and reservation state model |
| RLRS-F-013 | Scheduling Engine (maintenance blocks) |
| RLRS-F-014 to RLRS-F-016 / RLRS-NF-003, NF-004 | Notification Service, EmailJob, Job worker |
| RLRS-F-017 / RLRS-SR-005 | Record-level authorization on equipment detail |
| RLRS-F-019 | Reporting Service |
| RLRS-F-021 / RLRS-SR-007 | Audit Service |
| RLRS-NF-001 | Indexes on (equipment_id, start_at), query design, performance test suite |
| RLRS-NF-007 | Layered modular architecture and scheduling-engine unit tests |

### 11.1 Risks and Mitigations

| Risk | Mitigation |
|---|---|
| Double booking under concurrent requests | Database-level exclusion constraint or row lock; concurrency test in CI. |
| Off-by-one and time-zone errors at slot edges | Half-open ranges, UTC storage, and boundary tests for back-to-back, midnight and operating-hour edges. |
| Reminders sent twice, late or not at all | EmailJob table with unique reminder per reservation, row locking, retries, and a sent-time check. |
| Email provider limits or blocked SMTP in college network | Use a local mail catcher in development; document provider settings; keep in-app reservation list as the source of truth. |
| Merge conflicts across four contributors | Assign clear module ownership (UI, API, engine, worker), use feature branches and pull requests, integrate frequently. |
| Scope expansion (SSO, iCal, waitlists, SMS) | Prioritize the core browse/book/cancel/remind flow; label integrations as future work. |

### 11.2 Open Issues

- Confirm the final web framework, database, calendar library and job scheduler from the repository.
- Confirm default booking rules (advance window, maximum duration, maximum active bookings, cancellation cut-off) with the lab in-charge.
- Confirm whether faculty bookings can override student bookings or only use free slots.
- Confirm whether a waitlist for fully booked slots is required in the first release.
- Populate the final component/class/sequence diagrams using the actual module names before submission.